SKALA · 2026 하반기

sLLM 과 Fine Tuning

Day 1 실습 서브 노트

작업을 진행하며 궁금한 점, 의문인 점을 인용박스에 정리하면서 노트를 작성하였습니다.

이름 **김민정**

사번 **p282**

소속 **판교캠퍼스**

작성일 2026-08-30

Qwen2.5-1.5B-Instruct · BAAI/bge-m3 · FAISS · PEFT LoRA | Apple Silicon Mac (MPS / FP32)

목차

Chap 1	이해관계자 가치 (Pain Point)
Chap 2	Knowledge Asset 과 sLLM 활용
Chap 3	Mini Knowledge Base 구성도
Chap 4	PEFT(LoRA) 적용 전·후 Parameter 비교
Chap 5	SFT 적용 방향 및 다음 실습과의 연결
Chap 6	남은 의문과 다음 실습
부록	검색 점수로 Knowledge Base 밖 질문을 걸러낼 수 있을까

실습 환경

장비	Apple Silicon Mac 24GB · MPS / FP32
Python	3.11
패키지	`torch 2.13.0` `transformers 5.14.1` `peft 0.20.0` `faiss-cpu 1.15.0` `sentence-transformers 5.7.0`
모델	`Qwen/Qwen2.5-1.5B-Instruct` (생성) `BAAI/bge-m3` (임베딩)
노트북	`knowledge_asset_demo_faiss_bge_m3.ipynb` `peft_lora_parameter_compare_cache.ipynb`

Chap 1. 이해관계자 가치 (Pain Point)

> 요구사항
>
> 범용 LLM 을 기업·기관의 특정 업무에 그대로 적용할 때 발생할 수 있는 한계를 정의합니다. 사내 규정, 업무 지식, 전문 용어와 같은 고유한 Knowledge Asset 을 범용 모델이 충분히 알지 못하는 상황을 예로 들어, sLLM 이 왜 필요하고 특정 도메인에 모델을 특화하는 것이 어떤 이해관계자에게 어떤 가치를 제공할 수 있는지 제시합니다.

사전 학습 모델은 인터넷을 다 읽고 자란 신입사원 같습니다. 상식이나 언어 능력은 좋은데 회사 안에서만 통하는 규칙은 배운 적이 없습니다. 실습에 나온 가상 회사 Human AI Corporation 을 예로 들면 이런 것들입니다.

- 교육비 지원 제도 코드가 `HAI-EDU-2026` 이고 1인당 연간 한도가 120만원이라는 사실
- 교육비 신청은 시작일 14일 전까지, 승인은 팀장에서 HR 검토를 거쳐 최종 승인으로 간다는 절차
- 수료율이 80% 에 못 미치면 지원 대상에서 빠진다는 예외

문제는 모델이 이걸 모른다는 게 아니라, 모르면서도 답을 만들어 낸다는 데 있습니다. 사내 규정을 모르면 모른다고 하면 되는데 실제로는 일반적인 기업 관행을 끌어와서 그럴듯한 문장을 뱉습니다. 받는 사람 입장에서는 결론이 나왔으니 회사 규정인 줄 알고 넘어가게 됩니다. 틀렸다는 것보다 틀린 걸 읽는 사람이 알아챌 수 없다는 게 진짜 문제입니다. 결국 담당 부서에 한 번 더 확인하는 걸로 끝나면 챗봇을 쓴 의미가 없어집니다.

근거를 대고, 정해진 절차 순서를 지키고, 없는 건 없다고 답하는 일은 프롬프트만으로 안정적으로 만들기 어렵습니다. 그래서 밖으로 내보내기 어려운 사내 규정이나 내부 절차를 다루려면 sLLM 이 필요합니다. 이 지식은 답변할 때 검색으로 붙이거나(RAG), SFT·LoRA 로 모델의 답변 방식 자체를 바꾸는 두 가지 방법으로 씁니다.

1-1 이해관계자별 가치

이해관계자	범용 LLM 의 불편	도메인에 맞춘 sLLM이 줄 수 있는 것
현업 직원	규정을 매번 담당 부서에 물어봄, 답이 사람마다 다름	사내 규정 기준으로 일관된 답을 즉시 받음
HR·총무 담당자	같은 질문에 반복 응대, 문의가 병목	단순 문의는 모델이 걸러주고 예외 케이스만 처리
보안·법무	외부 API에 사내 문서를 태우기 부담	1.5B급 작은 모델이라 사내망·자체 GPU에서 운용 가능
운영·인프라	대형 모델 상시 호출 비용	작은 모델 + 가벼운 어댑터로 비용을 낮춤
AI 담당 조직	업무별로 모델을 따로 두면 관리 불가	Base 1개 + 용도별 Adapter 교체로 다중 업무 대응

> **왜 sLLM 일까**
>
> 처음엔 그냥 큰 모델 쓰면 안 되나 싶었는데, 사내 규정 같은 자료는 외부 API 로 보내기가 어렵습니다. 사내망에서만 돌려야 하는 조직이라면 모델을 자체 GPU 에 올릴 수 있느냐가 곧 도입 가능 여부가 되고, 그러면 모델 크기부터 현실적인 선에서 잡아야 합니다. 그래서 1.5B 를 골랐습니다.
>
> 파인튜닝도 같은 이유입니다. 15억 파라미터급이라 LoRA 로 노트북에서도 학습이 돌아갔고, 오늘 실습이 그걸 직접 보여준 셈입니다. HR 규정처럼 범위가 좁고 정답이 문서에 고정돼 있는 업무라면 초대형 모델의 범용 추론력까지는 필요하지 않습니다.
>
> 대신 작은 모델은 조건을 따지거나 숫자를 비교하는 데 약합니다. 이번 실습의 RAG 오답(5장)도 여기서 나왔고, 그래서 SFT 로 답변 습관을 잡아 줄 필요가 생깁니다.

Chap 2. Knowledge Asset 과 sLLM 활용

> 요구사항
>
> 기업이 보유한 사내 문서, 업무 규정, 매뉴얼, 전문 지식 등의 Knowledge Asset 을 sLLM 에서 어떻게 활용할 수 있는지 설명합니다. 오늘 실습한 BGE-M3 Embedding 과 FAISS Semantic Search 를 중심으로, 사용자 질문과 관련된 Knowledge 를 검색하고 이를 Base Qwen2.5-1.5B-Instruct 가 활용하여 답변하는 과정을 정리합니다.

사내 문서와 업무 규정, 매뉴얼 같은 것들이 Knowledge Asset 입니다. 이걸 전부 모델에 학습시켜 넣을 필요는 없습니다. 오늘은 RAG 방식으로, 자료를 모델 밖에 두고 필요할 때만 꺼내 쓰는 구조를 다뤘습니다.

BGE-M3 가 문서와 질문을 숫자 벡터로 바꾸고, FAISS 가 그중에서 질문과 가까운 문서를 찾습니다. 찾은 내용만 Qwen 에 넘겨주면 회사 규정을 모르는 모델도 그 범위 안에서는 답할 수 있습니다.

2-1 BGE-M3 Embedding 이 뭘까

Embedding 은 글을 컴퓨터가 비교할 수 있는 숫자 벡터로 바꾸는 작업입니다. 오늘 쓴 모델은 `BAAI/bge-m3` 입니다.

- Qwen 처럼 답을 만들지 않고, 문장의 뜻을 숫자로 표현하는 일만 합니다. 생성 모델과 임베딩 모델은 역할이 따로 있습니다.
- 이름의 M3 는 다국어(Multi-Linguality), 여러 검색 방식 지원(Multi-Functionality), 짧은 문장부터 긴 문서까지 처리(Multi-Granularity)를 뜻합니다. 한국어 사내 규정을 따로 학습시키지 않고 바로 벡터로 만들 수 있었던 이유입니다.
- 실습에서는 규정 6건과 사용자 질문을 각각 넣어 문서당 1024차원 벡터를 얻었습니다.
- `normalize_embeddings=True` 로 벡터 길이를 1 로 맞춰 두면, 뒤에서 내적만으로 유사도를 볼 수 있습니다. 코사인 유사도와 같은 결과가 나옵니다.

비유를 통해 이해 해보자면, 문서마다 지도 위에 좌표를 하나씩 찍는 것에 가깝습니다. "재택근무는 주 2회" 와 "집에서 일해도 되나요" 는 단어가 달라도 뜻이 가까우면 좌표가 가깝게 찍힙니다. Semantic Search 는 이걸 전제로 합니다.

2-2 FAISS Semantic Search 가 뭘까

FAISS 는 Facebook AI Similarity Search 의 줄임말로, 벡터를 저장해 두고 가장 비슷한 벡터를 빠르게 찾아 주는 라이브러리입니다. Semantic Search 는 "재택" 이라는 글자가 있는지 보는 키워드 매칭이 아니라, 임베딩 공간에서 뜻이 가까운 것을 찾는 검색입니다.

> **왜 IndexFlatIP 를 썼을까**
>
> FAISS 에는 인덱스 타입이 여러 개 있고 문서 규모에 따라 고르는 게 달라집니다. `IndexFlatIP` 는 저장된 벡터를 전부 하나씩 비교하는 방식이라 정확도가 100% 인 대신 문서 수에 비례해 느려집니다. 오늘처럼 6건이면 이게 제일 낫습니다. 반대로 `IndexIVFFlat` 이나 `IndexHNSW` 는 대충 비슷한 것만 빠르게 찾는 방식이라 수십만 건 규모에서 속도를 얻는 대신 몇 건은 놓칩니다.
>
> 그러니까 오늘 `IndexFlatIP` 를 쓴 건 문서가 적어서입니다. 사내 규정 전체로 늘리면 인덱스 타입을 다시 골라야 합니다. 정확도와 속도 사이의 트레이드오프는 결국 문서 규모가 정하는 것 같습니다.

2-3 오늘 실습에서 Knowledge Asset 을 활용한 방식

HR 규정 6건으로 Mini Knowledge Base 를 만들어 두고 썼습니다. 교육비 관련 4건, 리프레시 휴가 1건, 재택근무 1건이고 각 문서는 `id`, `title`, `content` 로 되어 있습니다.

2-4 준비 단계 (한 번만 실행)

1단계. Knowledge Documents

규정 6건을 `id`, `title`, `content` 형태로 정의했습니다. HR-EDU-001 은 제목이 "AI 교육비 지원 한도" 이고 본문에 `HAI-EDU-2026` 코드와 연간 120만원이 들어 있습니다. 이 상태로는 질문과 유사도를 비교할 수 없어서 다음 단계에서 벡터로 바꿉니다.

2단계. BGE-M3 로 벡터 만들기

각 문서의 `title + content` 를 BGE-M3 에 넣었습니다. 문서당 1024차원 벡터가 나왔고 `normalize_embeddings=True` 로 길이를 1 로 맞췄습니다. 뜻이 비슷한 문장일수록 벡터 방향이 가까워집니다.

knowledge_asset_demo_faiss_bge_m3.ipynb · Cell 7 (Embedding)

In [15]:

```
knowledge_texts = [
    f"{doc['title']}\n{doc['content']}"
    for doc in knowledge_documents
]

knowledge_embeddings = embedding_model.encode(
    knowledge_texts,
    normalize_embeddings=True,
)

knowledge_embeddings = np.asarray(
    knowledge_embeddings,
    dtype="float32",
)

print("Embedding 완료")
print("문서 수        :", knowledge_embeddings.shape[0])
print("Embedding 차원 :", knowledge_embeddings.shape[1])
```

※ 코드의 설명 주석은 지면상 생략했습니다.

Out:

```
Embedding 완료
문서 수        : 6
Embedding 차원 : 1024
```

BGE-M3 Embedding 실행 결과 — 문서 6건, Embedding 차원 1024

> **1024 라는 숫자는 어디서 나온 걸까**
>
> 출력에 `Embedding 차원 : 1024` 가 찍히는데, 제가 이 값을 지정한 적이 없어서 궁금했습니다. 찾아보니 BGE-M3 모델 자체가 1024차원으로 학습된 모델이라, 어떤 문장을 넣든 항상 1024개 숫자가 나오는 구조였습니다. 문장이 길든 짧든 결과 벡터 길이는 같습니다.
>
> 생각해 보면 당연합니다. 길이가 제각각이면 서로 내적을 할 수가 없습니다. 비교하려면 자릿수부터 같아야 합니다.
>
> 그래서 4단계에서 질문을 벡터로 만들 때도 같은 BGE-M3 를 써야 한다는 말이 이해됐습니다. 모델이 다르면 똑같이 1024차원이어도 각 자리가 뜻하는 바가 달라서 숫자만 맞고 의미는 어긋납니다.

3단계. FAISS Index 에 저장

벡터 6개를 `IndexFlatIP` 에 넣었습니다. FAISS 에는 벡터만 저장되고 원문은 들어가지 않아서, `id` 로 매핑해 두고 검색 후에 문서를 되찾아 옵니다. 6건에 1024차원, 4바이트씩이니 인덱스 크기는 24KB 정도입니다.

knowledge_asset_demo_faiss_bge_m3.ipynb · Cell 8 (FAISS Index)

In [17]:

```
embedding_dimension = knowledge_embeddings.shape[1]

faiss_index = faiss.IndexFlatIP(
    embedding_dimension
)

faiss_index.add(
    knowledge_embeddings
)

print("FAISS Index 구축 완료")
print("저장된 Vector 수:", faiss_index.ntotal)
```

※ 코드의 설명 주석은 지면상 생략했습니다.

Out:

```
FAISS Index 구축 완료
저장된 Vector 수: 6
```

FAISS IndexFlatIP 구축 — 저장된 Vector 수 6

> **FAISS 에 문서를 넣었는데 왜 원문이 안 보일까**
>
> 코드를 다시 보니 `faiss_index.add()` 에 넣은 게 `knowledge_documents` 가 아니라 `knowledge_embeddings` 였습니다. FAISS 에는 숫자 벡터만 들어가고 제목이나 본문은 저장되지 않습니다.
>
> 그래서 검색 결과로 돌아오는 것도 문서가 아니라 몇 번째 벡터인지를 알려 주는 `indices` 뿐이고, `search_knowledge()` 안에서 그 번호로 `knowledge_documents[doc_index]` 를 다시 꺼내 옵니다.
>
> 처음엔 번거로워 보였는데 벡터 저장소와 원문 저장소를 나눠 두는 게 오히려 자연스러운 구조 같습니다. 실무라면 원문은 DB 에 있을 테고 FAISS 는 몇 번 문서가 가까운지만 알려 주면 되니까요.

2-5 질문 처리 단계 (질문마다 반복)

4단계. 질문을 벡터로

질문을 같은 BGE-M3 에 넣어 1024차원 벡터 하나를 만듭니다. 문서와 질문은 반드시 같은 모델을 써야 합니다.

5단계. Semantic Search

질문 벡터와 저장된 벡터 6개의 내적을 계산해 유사도 점수를 구하고, 높은 순으로 2건을 가져옵니다. 교육비 한도 질문에서는 HR-EDU-001 이 0.7812, HR-EDU-004 가 0.6470 으로 나왔습니다.

knowledge_asset_demo_faiss_bge_m3.ipynb · Cell 10 (Semantic Search)

In [21]:

```
question = (
    "AI 교육비는 연간 얼마까지 "
    "지원받을 수 있나요?"
)

search_results = search_knowledge(
    question,
    top_k=2,
)

print("[Question]")
print(question)

for rank, result in enumerate(
    search_results,
    start=1,
):
    print(
        f"\n[TOP {rank}] "
        f"score={result['score']:.4f}"
    )

    print(
        "ID     :",
        result["id"],
    )

    print(
        "Title  :",
        result["title"],
    )

    print(
        "Content:",
        result["content"],
    )
```

Out:

```
[Question]
AI 교육비는 연간 얼마까지 지원받을 수 있나요?

[TOP 1] score=0.7812
ID     : HR-EDU-001
Title  : AI 교육비 지원 한도
Content: Human AI Corporation의 AI 교육비 지원 제도 코드는 HAI-EDU-2026이다. 임직원 1인당 연간 최대 지원 금액은 120만원이다.

[TOP 2] score=0.6470
ID     : HR-EDU-004
Title  : AI 교육비 지원 제외 조건
Content: 교육 수료율이 80% 미만이면 교육비 지원 대상에서 제외된다. 사전 승인을 받지 않은 교육은 원칙적으로 소급 지원하지 않는다.
```

Semantic Search 실행 — Top-1 0.7812 / Top-2 0.6470

> **score 가 0.78 이면 높은 걸까 기준이 뭐지**
>
> 점수가 나오긴 하는데 높은 건지 낮은 건지 판단할 기준이 없어서 여러 질문을 돌려 보고 값을 모아 봤습니다. Top-1 이 Knowledge Base 안에 있을 때는 0.6913 에서 0.8096, Knowledge Base 밖일 때는 0.5647 이었습니다.
>
> 정규화를 했으니 이 값은 코사인 유사도라 이론상 -1 에서 1 사이인데 실제로 관측된 건 0.45 에서 0.81 구간이었습니다. 절대 기준이 정해져 있다기보다 내 데이터에서 분포를 보고 정하는 값 같습니다.
>
> 한 가지 걸린 건 Top-2 점수가 Knowledge Base 안 질문에서도 0.46 까지 떨어진다는 겁니다. Top-2 라고 둘 다 관련 문서인 건 아닌데 지금 코드는 무조건 2건을 프롬프트에 넣습니다. 관련 없는 문서를 억지로 근거라고 넣어 주는 셈이라 이게 괜찮은지 의문이 들었습니다.

6단계. System Prompt 조립

찾은 2건의 원문을 `id` 로 꺼내 System Prompt 의 `[Retrieved Knowledge]` 블록에 넣습니다. 6건 전체가 아니라 검색된 2건만 들어갑니다. 여기에 "검색된 내용만 근거로 답하고 없으면 없다고 하라" 는 지시를 함께 붙입니다.

7단계. Qwen 답변 생성

조립한 프롬프트와 질문을 Qwen2.5-1.5B-Instruct 에 넣으면 답이 나옵니다. Qwen 은 회사 규정을 배운 적이 없지만 6단계에서 넣어 준 2건을 근거로 "연간 최대 120만원" 이라고 답합니다.

knowledge_asset_demo_faiss_bge_m3.ipynb · Cell 12 (Search + Qwen)

In [25]:

```
search_results = search_knowledge(
    question,
    top_k=2,
)

answer = generate_answer(
    question=question,
    retrieved_documents=search_results,
)

print("[Question]")
print(question)

print("\n[Retrieved Knowledge]")

for result in search_results:
    print(
        f"- {result['id']} | "
        f"{result['title']} | "
        f"score={result['score']:.4f}"
    )

print("\n[Qwen Answer]")
print(answer)
```

Out:

```
[Question]
AI 교육비는 연간 얼마까지 지원받을 수 있나요?

[Retrieved Knowledge]
- HR-EDU-001 | AI 교육비 지원 한도 | score=0.7812
- HR-EDU-004 | AI 교육비 지원 제외 조건 | score=0.6470

[Qwen Answer]
AI 교육비는 연간 120만원까지 지원받을 수 있습니다.
```

검색 System Prompt 주입 Qwen 답변 생성

> **답을 Qwen 이 아는 걸까**
>
> 120만원이라는 답이 나왔지만 Qwen 이 Human AI Corporation 을 알아서 나온 게 아닙니다. 바로 위 `[Retrieved Knowledge]` 에 그 문장이 통째로 들어가 있어서 나온 겁니다.
>
> 확인하고 싶어서 검색 없이 같은 질문을 던지면 어떻게 될지 생각해 봤는데, `HAI-EDU-2026` 은 실제로 없는 가상의 코드라 Base 모델이 맞힐 방법이 없습니다. 답의 출처가 모델이 아니라 프롬프트라는 게 중요하다고 생각했습니다.
>
> 그래서 이 구조에서는 검색이 틀리면 답도 같이 틀립니다. 모델을 좋은 걸 써도 검색 단계에서 엉뚱한 문서를 주면 그대로 끌려갑니다. RAG 에서 제일 약한 고리인 것 같습니다.

2-6 실습 결과로 확인한 점

질문	검색 Top-1 (score)	Qwen 답변
AI 교육비는 연간 얼마까지 지원받을 수 있나요?	HR-EDU-001 지원 한도 (0.78)	연간 최대 **120만원 정답**
AI 교육비 신청은 교육 시작 며칠 전까지 해야 하나요?	HR-EDU-002 신청 기한 (0.81)	**14일 전 정답**
재택근무는 일주일에 몇 번 가능한가요?	HR-WORK-001 재택근무 (0.77)	주 최대 **2회 정답**
해외 AI 컨퍼런스 항공료를 전액 지원하나요?	HR-EDU-001 (0.56, Knowledge Base 밖)	"Knowledge Base에서 확인할 수 없습니다" **거절**

Knowledge Base 안 질문은 유사도 0.69 에서 0.81 사이로 관련 문서를 잘 찾았습니다. Knowledge Base 에 없는 항공료 질문에서는 FAISS 가 가장 가까운 문서를 돌려줬지만 점수가 0.56 으로 낮았고, Qwen 은 지어내지 않고 답할 수 없다고 했습니다.

"집에서 일해도 괜찮은가요?" 처럼 표현이 다른 질문도 재택근무 규정을 찾아왔습니다. 키워드가 안 겹쳐도 뜻이 비슷하면 검색되는 게 Semantic Search 의 장점입니다.

knowledge_asset_demo_faiss_bge_m3.ipynb · Cell 14 (KB 밖 질문)

In [29]:

```
unknown_question = (
    "Human AI Corporation은 해외 AI 컨퍼런스 참석 시 "
    "항공료를 전액 지원하나요?"
)

unknown_results = search_knowledge(
    unknown_question,
    top_k=2,
)

print("[Unknown Question]")
print(unknown_question)

print("\n[Retrieved Knowledge]")

for result in unknown_results:
    print(
        f"- {result['id']} | "
        f"{result['title']} | "
        f"score={result['score']:.4f}"
    )

unknown_answer = generate_answer(
    question=unknown_question,
    retrieved_documents=unknown_results,
)

print("\n[Qwen Answer]")
print(unknown_answer)
```

Out:

```
[Unknown Question]
Human AI Corporation은 해외 AI 컨퍼런스 참석 시 항공료를 전액 지원하나요?

[Retrieved Knowledge]
- HR-EDU-001 | AI 교육비 지원 한도 | score=0.5647
- HR-EDU-003 | AI 교육비 승인 절차 | score=0.4470

[Qwen Answer]
현재 Knowledge Base에서 해당 내용을 확인할 수 없습니다.
```

Knowledge Base 밖 질문 — Top-1 score 0.5647, Qwen 은 거절

> **FAISS 가 걸러낸 걸까**
>
> 답변은 "확인할 수 없습니다" 로 잘 나왔는데 출력을 다시 보다가 이상한 걸 발견했습니다. FAISS 는 거절을 한 적이 없습니다. 점수 0.5647 로 HR-EDU-001 을 멀쩡히 Top-1 으로 돌려줬고 검색 단계는 평소처럼 2건을 그대로 넘겼습니다.
>
> 거절을 판단한 건 Qwen 이고, 그것도 System Prompt 에 없으면 없다고 하라고 적어 둔 덕분입니다. 모델이 그 지시를 안 따르는 날에는 그냥 지어낼 수도 있다는 뜻입니다. 실제로 수료율 70% 질문(5장)에서 지시를 안 따르는 걸 봤기 때문에 더 그렇게 느꼈습니다.
>
> 그런데 점수를 보면 Knowledge Base 안(0.69~0.81)과 Knowledge Base 밖(0.56)이 꽤 벌어져 있습니다. FAISS 는 이미 신호를 숫자로 보내고 있었는데 코드가 그걸 안 쓰고 있었던 겁니다. 이 생각이 부록 실험으로 이어졌습니다.

2-7 범용 모델이 회사 지식을 몰라도 대응되는 이유

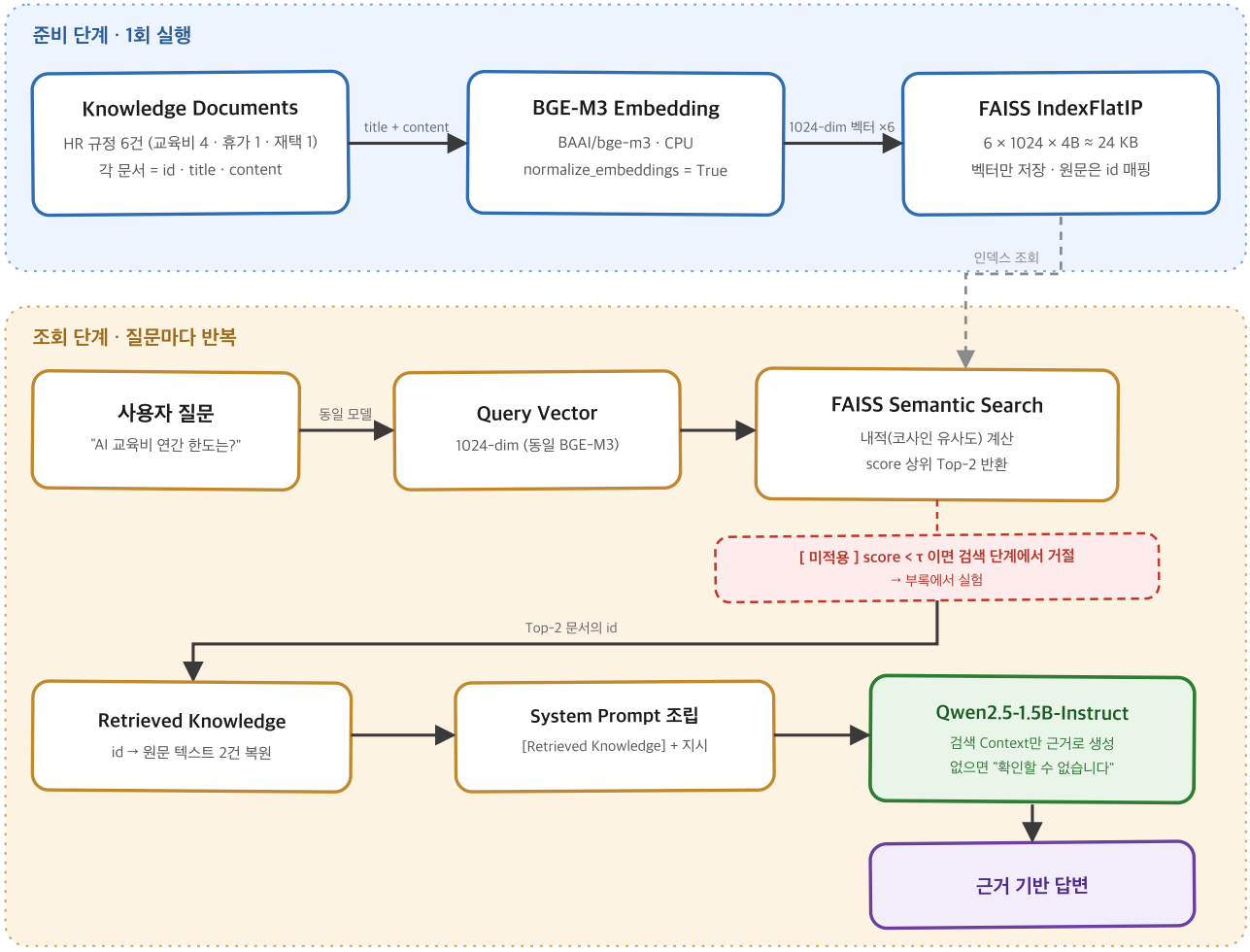
Qwen2.5-1.5B-Instruct 는 범용 모델이라 `HAI-EDU-2026` 이나 연간 120만원 같은 회사 고유 지식이 학습 데이터에 없습니다. 그래도 자료를 밖에서 찾아 프롬프트로 넣어 주면 그 범위 안에서는 근거 있는 답이 나옵니다. 규정이 바뀌면 인덱스만 다시 만들면 되고 모델은 건드리지 않아도 됩니다.

> **그렇다면 실무 확장 시엔 뭐가 달라질까**
>
> 오늘은 문서 6건을 통째로 임베딩했는데 실제 사내 규정은 문서 하나가 수십 페이지입니다. 문서 전체를 벡터 하나로 만들면 뜻이 평균으로 뭉개져서 검색이 잘 안 될 겁니다. 조항이나 섹션 단위로 잘라야 합니다. SFT 데이터셋이 규정을 97개 섹션으로 쪼개 둔 것과 같은 발상입니다.
>
> 문서 조각이 늘면 Top-2 로는 근거가 모자랄 수 있으니 K 를 올리고 Reranker 를 붙이게 되고, 인덱스도 `IndexFlatIP` 에서 `IndexHNSW` 같은 걸로 바꾸게 됩니다.

Chap 3. Mini Knowledge Base 구성도

> 요구사항
>
> 오늘 실습한 Knowledge 활용 과정을 한눈에 볼 수 있도록 간단한 아키텍처를 구성합니다. Knowledge Documents 가 BGE-M3 를 통해 Embedding Vector 로 변환되고 FAISS Index 에 저장되는 과정과, 사용자 질문이 Embedding 된 후 Semantic Search 를 통해 관련 Knowledge 가 검색되어 Qwen 에 전달되는 흐름을 포함합니다.

인덱스를 만드는 준비 단계와 질문에 답하는 조회 단계로 구분됩니다.



Mini Knowledge Base 파이프라인 — 준비 단계(파랑)는 1회, 조회 단계(주황)는 질문마다 반복

구성 요소	역할	오늘 실습 설정
Knowledge Documents	사내 규정·매뉴얼 등 Knowledge Asset 원본	HR 규정 6건 (`id`, `title`, `content`)
BGE-M3	문서·질문을 의미 벡터로 변환 (Embedding)	`BAAI/bge-m3`, 1024차원, CPU 실행
FAISS Index	Embedding Vector 저장 및 유사도 검색	`IndexFlatIP`, 전수 내적 방식
Semantic Search	질문과 의미가 가까운 Knowledge 검색	Top-2, score = 내적(코사인 유사도)
Qwen2.5-1.5B-Instruct	검색된 Context를 바탕으로 답변 생성	Base 모델, System Prompt에 Knowledge 주입

Chap 4. PEFT(LoRA) 적용 전·후 Parameter 비교

> 요구사항
>
> Base Qwen2.5-1.5B-Instruct 에 PEFT(LoRA)를 적용하기 전과 후의 Parameter 구조를 비교합니다. 전체 Parameter 와 실제 학습 가능한 Trainable Parameter 가 어떻게 달라지는지 실습 결과를 바탕으로 정리하고, 전체 모델 Parameter 를 모두 학습하지 않고 일부 추가 Parameter 만 학습하는 LoRA 방식이 왜 효율적인 Fine-Tuning 방법인지 설명합니다.

4-1 적용 전과 후 비교

워밍업 설정은 `r=8`, `lora_alpha=16`, `target_modules=["q_proj", "v_proj"]`, `lora_dropout=0.05`, `bias="none"` 입니다.

구분	전체 Parameter	Trainable Parameter	Trainable 비율
LoRA 적용 전 (Base)	1,543,714,304 (약 15.4억)	1,543,714,304	100.00%
LoRA 적용 후 (q_proj, v_proj / r=8)	1,544,803,840	1,089,536 (약 109만)	0.0705%

Full Fine-Tuning 은 Base 모델의 가중치 15.4억 개를 전부 업데이트합니다. LoRA 를 적용하면 기존 파라미터는 `requires_grad=False` 로 얼어붙고 새로 붙인 어댑터만 학습돼서, 학습 대상이 0.07% 로 줄어듭니다.

peft_lora_parameter_compare_cache.ipynb · Cell 6 (LoRA 적용 전)

In [14]:

```
def count_parameters(model):

    total_params = sum(
        p.numel()
        for p in model.parameters()
    )

    trainable_params = sum(
        p.numel()
        for p in model.parameters()
        if p.requires_grad
    )

    return total_params, trainable_params

total_before, trainable_before = count_parameters(model)

print("===== LoRA 적용 전 =====")
print(f"전체 파라미터      : {total_before:,}")
print(f"학습 가능 파라미터 : {trainable_before:,}")
print(f"학습 가능 비율     : {trainable_before / total_before * 100:.4f}%")
```

※ 코드의 설명 주석은 지면상 생략했습니다.

Out:

```
===== LoRA 적용 전 =====
전체 파라미터      : 1,543,714,304
학습 가능 파라미터 : 1,543,714,304
학습 가능 비율     : 100.0000%
```

LoRA 적용 전 — 전체 1,543,714,304 / 학습 가능 100%

peft_lora_parameter_compare_cache.ipynb · Cell 7-8 (LoRA 설정 → 적용 후)

In [16]:

```
lora_config = LoraConfig(

    r=8,

    lora_alpha=16,

    target_modules=[
        "q_proj",
        "v_proj"
    ],

    lora_dropout=0.05,

    bias="none",

    task_type="CAUSAL_LM"
)

model = get_peft_model(
    model,
    lora_config
)

print("LoRA 적용 완료")
```

※ 코드의 설명 주석은 지면상 생략했습니다.

Out:

```
LoRA 적용 완료
```

In [18]:

```
total_after, trainable_after = count_parameters(model)

print("===== LoRA 적용 후 =====")
print(f"전체 파라미터      : {total_after:,}")
print(f"학습 가능 파라미터 : {trainable_after:,}")
print(f"학습 가능 비율     : {trainable_after / total_after * 100:.4f}%")

print("\n===== PEFT 공식 출력 =====")
model.print_trainable_parameters()
```

Out:

```
===== LoRA 적용 후 =====
전체 파라미터      : 1,544,803,840
학습 가능 파라미터 : 1,089,536
학습 가능 비율     : 0.0705%

===== PEFT 공식 출력 =====
trainable params: 1,089,536 || all params: 1,544,803,840 || trainable%: 0.0705
```

LoRA 설정 및 적용 후 — 학습 가능 1,089,536 / 0.0705%

> **전체 파라미터가 오히려 늘었는데 왜 "효율적"이라고 할까**
>
> 두 출력을 나란히 놓고 보다가 헷갈렸습니다. 전체 파라미터가 줄어든 게 아니라 늘었기 때문입니다. 그래서 두 수를 직접 빼 봤더니 `1,544,803,840 − 1,543,714,304 = 1,089,536` 으로, 늘어난 파라미터 수와 학습 가능한 파라미터 수가 정확히 같았습니다. 새로 붙인 건 전부 학습 대상이고 원래 있던 건 하나도 안 건드린다는 뜻이었습니다.
>
> 효율적이라는 말은 모델이 작아졌다는 게 아니라 역전파할 대상이 줄었다는 의미였습니다. 모델은 오히려 조금 무거워졌고 학습이 가벼워졌습니다.

4-2 1,089,536개가 어떻게 나오는지

LoRA 는 지정한 모듈 옆에 `lora_A`, `lora_B` 라는 작은 행렬 두 개를 붙입니다. 28개 레이어에 모듈 2개씩, 각각 A 와 B 가 있으니 텐서는 112개가 생깁니다.

모듈	lora_A	lora_B	합계
q_proj	[8, 1536] = 12,288	[1536, 8] = 12,288	24,576
v_proj	[8, 1536] = 12,288	[256, 8] = 2,048	14,336
레이어 합계			**38,912**

레이어 하나가 38,912개이고 28개 층이니 곱하면 1,089,536 이 나옵니다.

peft_lora_parameter_compare_cache.ipynb · Cell 9 (실제 학습 대상 파라미터)

```
In [20]:
for name, param in model.named_parameters():
    if param.requires_grad:
        print(f"{name:<90} "
              f"shape={str(list(param.shape)):<18} "
              f"params={param.numel():,}")
```

※ 코드의 설명 주석은 지면상 생략했습니다.

```
Out:
===== 실제 학습되는 파라미터 =====
base_model.model.model.layers.0.self_attn.q_proj.lora_A.default.weight                  shape=[8, 1536]          params=12,288
base_model.model.model.layers.0.self_attn.q_proj.lora_B.default.weight                  shape=[1536, 8]          params=12,288
base_model.model.model.layers.0.self_attn.v_proj.lora_A.default.weight                  shape=[8, 1536]          params=12,288
base_model.model.model.layers.0.self_attn.v_proj.lora_B.default.weight                  shape=[256, 8]           params=2,048
base_model.model.model.layers.1.self_attn.q_proj.lora_A.default.weight                  shape=[8, 1536]          params=12,288
base_model.model.model.layers.1.self_attn.q_proj.lora_B.default.weight                  shape=[1536, 8]          params=12,288
base_model.model.model.layers.1.self_attn.v_proj.lora_A.default.weight                  shape=[8, 1536]          params=12,288
base_model.model.model.layers.1.self_attn.v_proj.lora_B.default.weight                  shape=[256, 8]           params=2,048

...  (layers.2 ~ layers.27 동일 패턴, 총 112개 LoRA 텐서)
```

실제 학습되는 파라미터 목록 — lora_A / lora_B 의 shape 확인

> **v_proj 의 lora_B 만 왜 혼자 [256, 8] 일까**
>
> 출력을 보면 q_proj 는 A 와 B 가 둘 다 1536 인데 v_proj 는 lora_A 만 1536 이고 lora_B 는 256 이었습니다. 찾아보니 GQA(Grouped-Query Attention) 때문이었습니다. Qwen2.5-1.5B 는 Query head 가 12개인데 Key/Value head 는 2개만 둡니다. 그래서 Value 출력 차원이 2 곱하기 128 로 256 이고, 거기 붙는 lora_B 도 256 이 됩니다.
>
> LoRA 크기는 제가 정하는 값이 아니라 붙는 Linear 층의 shape 을 따라간다는 걸 알게 됐습니다. 제가 지정한 건 `r=8` 하나뿐이고 나머지 한쪽 변은 모델 구조가 정해 줍니다.
>
> `layers.0` 부터 `layers.27` 까지 같은 패턴이 28번 반복되는 것도 눈으로 봤습니다. `q_proj` 하나를 적었을 뿐인데 28개 층 전부에 각각 독립된 어댑터가 붙는다는 걸 알게 되었습니다.

4-3 LoRA 동작 원리

원본 가중치 W 는 그대로 두고, 옆에 `ΔW = B × A` 형태의 작은 행렬만 학습합니다.

```
출력 = W·x  +  (B·A)·x
       ↑          ↑
    Freeze     Trainable
  (기존 15.4억)  (LoRA ~109만)
```

A 는 `[r, in_features]`, B 는 `[out_features, r]` 크기이고, r 은 변화량을 표현할 방향의 개수 상한입니다.

> **저랭크 행렬이 뭘까, B 를 0으로 두는 이유는 뭘까**
>
> 파인튜닝에 필요한 가중치 변화는 원본 W 만큼 복잡하지 않고 몇 개 방향으로 표현할 수 있다는 가정이 저랭크 가설입니다. `r=8` 은 최대 8개 방향까지만 쓰겠다는 뜻입니다. 15.4억 개의 자유도가 필요한 게 아니라 "HR 규정 말투로 답한다" 정도의 변화는 낮은 차원의 조정이라고 보는 관점입니다.
>
> B 를 0 으로 두는 건 학습 시작 시점에 `B·A = 0` 이라 변화량이 0 이 되기 때문입니다. LoRA 를 붙인 직후의 모델은 Base 와 똑같은 출력을 냅니다. 어댑터를 붙였다는 이유만으로 성능이 망가지지 않고, 0 에서부터 변화량이 쌓입니다. 원본을 훼손하지 않는다는 원칙이 초기화 설계에까지 들어가 있었습니다.

4-4 설정을 바꾸면 얼마나 달라지나

같은 Base 모델에 rank 와 target_modules 를 바꿔 가며 확인했습니다.

설정	target_modules	r	Trainable	비율	Adapter 크기(FP32)
LoRA 적용 전	—	—	1,543,714,304	100.00%	—
워밍업 (오늘 기본)	q_proj, v_proj	8	**1,089,536**	**0.0705%**	4.16 MB
rank 축소	q_proj, v_proj	4	544,768	0.0353%	2.08 MB
rank 확대	q_proj, v_proj	16	2,179,072	0.1410%	8.31 MB
rank 확대	q_proj, v_proj	32	4,358,144	0.2815%	16.62 MB
Attention 전체	q, k, v, o_proj	8	2,179,072	0.1410%	8.31 MB
train_lora.py	q,k,v,o + gate,up,down	16	**18,464,768**	**1.1820%**	70.44 MB

rank 를 두 배로 키우면 학습 파라미터도 두 배가 되고, target_modules 를 늘리면 붙는 텐서 수만큼 늘어납니다.

> **어느 모듈에 붙일지는 어떻게 정할까**
>
> 표를 보다가 `q,v / r=16` 과 `q,k,v,o / r=8` 의 학습 파라미터가 2,179,072개로 똑같다는 걸 발견했습니다. 앞은 적은 모듈을 깊게, 뒤는 많은 모듈을 얕게 조정하는 셈이라 같은 예산을 깊이에 쓸지 넓이에 쓸지의 선택이었습니다. 파라미터 수만으로는 어느 쪽이 나은지 알 수 없습니다.
>
> 왜 하필 q_proj 와 v_proj 부터인지 찾아보니, LoRA 원 논문에서 같은 예산이면 Query 와 Value 에 적용하는 게 효율적이라고 보고했기 때문이었습니다. Query 는 무엇에 주목할지, Value 는 주목한 것에서 무엇을 꺼낼지를 정하니 답변 스타일 조정에 직접 관여합니다.
>
> 반면 `train_lora.py` 는 MLP 까지 확장합니다. 스타일을 넘어 AURORA 나 BLUE-7 같은 새 지식을 넣으려면 지식이 저장되는 MLP 층까지 건드려야 하기 때문으로 이해했습니다.

4-5 LoRA 가 효율적인 이유

학습해야 할 대상이 0.07% 밖에 안 되니 옵티마이저가 들고 있어야 할 상태값도 그만큼만 필요합니다. 얼어붙은 15.4억 개는 기울기를 저장하지 않아도 되어서 메모리가 크게 절약됩니다.

결과물도 전체 모델 6GB 가 아니라 어댑터 파일 하나입니다. 워밍업 설정이면 4MB 정도입니다. Base 모델 하나에 HR용, 법무용 어댑터를 갈아 끼우는 식으로 운영할 수 있습니다.

원본 가중치를 수정하지 않으니 모델이 원래 갖고 있던 언어 능력을 통째로 잃을 위험도 작습니다. 학습 데이터가 582건뿐인 상황에서 15억 개를 다 학습시키면 과적합이 나기 쉬운데, LoRA 는 적은 파라미터만 조정해서 그 위험을 줄입니다.

Chap 5. SFT 적용 방향 및 다음 실습과의 연결

> 요구사항
>
> 오늘 학습한 SFT(Supervised Fine-Tuning)의 개념을 바탕으로, 기업의 Knowledge Asset 과 업무 요구사항을 학습 데이터로 구성하여 모델을 특정 업무에 특화할 수 있는 방법을 정리합니다. 오늘 실습한 Knowledge 검색 방식과 SFT 의 차이를 간단히 비교하고, 다음 실습에서 Train·Validation Dataset 을 이용한 LoRA SFT 학습과 Evaluation 을 통해 Base 모델과 Fine-tuned 모델의 차이를 실제로 검증하게 된다는 점까지 제시합니다.

5-1 SFT 개념과 학습 데이터

SFT 는 "이 지시에 이 질문이 오면 이런 형식과 내용으로 답한다" 는 관계를 학습시키는 방식입니다. 사내 규정을 반영하려면 규정과 매뉴얼을 바탕으로 system, user, assistant 세 턴짜리 데이터를 만듭니다.

```
{
  "id": "section-075-train-01",
  "split": "train",
  "source_section_id": "section-075",
  "source_title": "1. AURORA 재택근무 등급",
  "messages": [
    {"role": "system",    "content": "당신은 Human AI Corporation의 인사 규정 안내 도우미입니다. (근거 우선, 고유 코드 정확히, 없으면 추측하지 말 것 ...)"},
    {"role": "user",      "content": "AURORA 재택근무 등급에 대한 규정을 설명해 주세요."},
    {"role": "assistant", "content": "결론:\n회사는 재택근무 가능 범위를 AURORA 등급으로 관리한다.\n\n적용 규정:\n- AURORA-A: 주 3일까지 ... - AURORA-X: 전면 금지\n\n위 규정만으로 판단하기 어려운 개별 상황은 인사팀에 확인해야 합니다."}
  ]
}
```

5-2 데이터셋을 뜯어보며 알게 된 것

설계 요소	데이터에서 관찰된 값	의도한 SFT 효과
페르소나·규범 고정	system 프롬프트 **1종**, 582건 100% 동일 (6개 원칙)	매 예시마다 같은 규범을 재확인 → instruction-following 안정화
토픽 노출 균형	**97개 섹션 × 6개 질문 = 582건** 균등 분포	특정 조항 과적합 방지, 규정 전 영역 균등 학습
출력 포맷 표준화	**576/582건**이 "결론 → 적용 규정" 2단 템플릿	두괄식 응답 습관을 구조적으로 학습, 장황화 억제
네거티브(거절) 샘플	**6/582건 (약 1%)** 을 거절형으로 구성	커버리지 밖 질문의 환각 억제, 절제된 에스컬레이션 학습
출처 추적성	`id` 에 `source_section_id` • 생성 순번 인코딩	오답 발생 시 어느 원문 섹션에서 왔는지 역추적 가능

> **거절 6건으로 뭐가 달라질까**
>
> 1장에서 정리한 문제, 그러니까 모르면서 그럴듯하게 답하는 걸 막는 장치가 데이터셋 안에서는 전체의 1% 인 negative sample 6건으로 들어가 있었습니다. `source_section_id` 가 null 이고 `source_title` 이 `unknown_policy` 로 표시된 것들입니다.
>
> 거절 샘플이 아예 없으면 모델은 어떤 질문이 와도 답을 만들어 내는 게 정답이라고 학습하게 됩니다. 환각을 줄이는 게 모델 크기가 아니라 데이터를 어떻게 짜느냐의 문제일 수 있다는 걸 알게 됐습니다.

학습 데이터에는 AURORA, BLUE-7, ORBIT-3 같은 회사 고유 코드와 "결론 다음에 적용 규정" 이라는 답변 형식, 근거 없이 단정하지 않는다는 규칙이 들어갑니다.

데이터셋	건수	구성	용도
`hr_sft_train.jsonl`	582	97 섹션 × 6 = 576 정상 + 6 거절	LoRA SFT 학습
`hr_sft_validation.jsonl`	97	섹션당 1건	학습 중 eval_loss 모니터링(과적합 감시)
`hr_eval.jsonl`	20	known_policy_paraphrase 15 + unknown_policy 5	학습 후 Base vs Fine-tuned 비교 (학습에 미사용)

5-3 RAG 와 SFT 의 차이

구분	Knowledge 검색 (오늘 실습)	SFT / LoRA (다음 실습)
지식의 위치	모델 **밖**. 질문 시점에 검색하여 프롬프트에 주입	모델 **안**. LoRA Adapter 가중치에 학습
바꾸는 것	모델은 그대로, **Context 만** 변경	모델의 **답변 습관·형식·판단 방식** 변경
규정 변경 시	문서 갱신 후 Embedding 재구축 (수 분)	데이터 수정 후 **재학습** (수 시간)
잘하는 것	최신 사실을 출처와 함께 제공	짧은 질문에도 회사 형식·코드·거절 규칙 준수
약한 것	검색 오류 전파, 작은 모델의 조건 판단 오류	방대한 새 사실 주입에 부적합, 재학습 비용
추론 비용	프롬프트가 길어져 토큰·지연 증가	프롬프트 짧음, 어댑터 오버헤드만 존재

5-4 오늘 확인한 RAG 의 한계

"수료율 70% 면 교육비 지원이 되나요" 라는 질문에서 FAISS 는 HR-EDU-004 를 0.69 로 정확히 찾았는데 Qwen 은 지원 가능하다고 틀린 답을 냈습니다. 근거는 제대로 전달됐는데 모델이 70 과 80 을 비교하는 데서 어긋난 경우입니다.

knowledge_asset_demo_faiss_bge_m3.ipynb · Cell 13 (질문 3 / 5)

In [27]:

```
test_questions = [
    ...,
    "수료율이 70%라면 교육비 지원을 받을 수 있나요?",
    ...,
]

for test_question in test_questions:
    retrieved = search_knowledge(test_question, top_k=2)
    answer    = generate_answer(question=test_question,
                                retrieved_documents=retrieved)
```

※ 5개 질문 중 3번째 결과만 발췌했습니다.

Out:

```
[Question 3]
수료율이 70%라면 교육비 지원을 받을 수 있나요?

[Retrieved]
- HR-EDU-004 | AI 교육비 지원 제외 조건 | 0.6913
- HR-EDU-001 | AI 교육비 지원 한도 | 0.4828

[Answer]
네, 수료율이 70%라면 교육비 지원을 받을 수 있습니다. 그러나 이 경우 지원금액은 120만원보다 적게 지급될 것입니다.
```

수료율 70% 질문 — 검색은 정확했으나 답변은 오답

> **근거를 눈앞에 두고도 틀리는 건 왜일까**
>
> 검색이 정확했는데 왜 답변이 오답인지 궁금했습니다. 검색된 Top-1 은 HR-EDU-004 "수료율 80% 미만이면 제외된다" 였고 그 문장이 프롬프트에 그대로 들어갔는데도 답은 "네, 받을 수 있습니다" 였습니다. 규정에 없는 "120만원보다 적게 지급" 이라는 말까지 덧붙였습니다.
>
> 검색은 제대로 했는데 답변이 이상하게 나왔다면, 이건 검색 실패가 아니라 추론 실패라고 판단했습니다. `70 < 80` 이라는 비교를 모델이 못 한 겁니다.
>
> 그러면 검색기를 아무리 고쳐도 이 오답은 안 사라집니다. Top-K 를 늘려도, 임계값을 걸어도, 가장 관련 있는 문서는 이미 찾았기 때문입니다.
>
> RAG 는 무엇을 보여줄지를 바꾸고 SFT 는 본 것을 어떻게 판단할지를 바꿉니다. 지금 문제는 "본 것을 어떻게 판단할지" 쪽에 있다고 생각했습니다.

그래서 RAG 와 SFT 는 서로 대체하는 관계가 아닙니다. 실무에서는 SFT 로 답하는 방식을 고정해 두고 RAG 로 그때그때 근거 문장을 붙입니다. 규정 개정처럼 사실이 자주 바뀌는 건 RAG 쪽이고, 답변 템플릿이나 거절 규칙처럼 잘 안 바뀌는 건 SFT 쪽입니다.

5-5 다음 실습에서 할 것

오늘은 LoRA 구조만 확인했고, 다음 실습에서는 실제로 가중치를 학습시키고 Base 와 비교합니다.

항목	설정
Base Model	Qwen2.5-1.5B-Instruct
학습 데이터	`hr_sft_train.jsonl` (582건)
검증 데이터	`hr_sft_validation.jsonl` (97건)
LoRA 설정	q/k/v/o + gate/up/down, r=16 (Trainable 약 1,846만, 1.18%)
권장 하이퍼파라미터	MAX_LENGTH=384, EPOCHS=2, LR=1e-4
실행 스크립트	`scripts/train_lora.py`
결과물	`models/hr-qwen-lora/` (Adapter 파일, 전체 모델 아님)

학습 중에 train loss 와 eval_loss 를 같이 보면서 과적합을 확인합니다.

항목	내용
평가 데이터	`hr_eval.jsonl` (20건, 학습에 미사용)
비교 대상	Base Qwen vs Fine-tuned Qwen (동일 질문)
평가 지표	Keyword Score, Token F1, Hallucination
확인 목적	SFT가 의도한 방향으로 모델 행동이 바뀌었는지 검증

평가 질문은 AURORA 재택근무 등급, BLUE-7 보안등급, 규정에 없는 질문 거절 같은 것들입니다. `must_include` 와 `must_not_include` 필드로 고유 코드가 들어갔는지와 없는 내용을 지어냈는지를 채점합니다.

Chap 6. 남은 의문과 다음 실습

6-1 더 확인해 보고 싶은 것

검색 점수 관련 질문은 오늘 실습 환경에서 바로 확인할 수 있을 것 같아 직접 돌려 봤고 결과는 부록에 정리했습니다.

- 검색 점수로 Knowledge Base 밖 질문을 걸러낼 수 있을까. 점수가 몇이든 무조건 Top-2 를 넣고 있는데 그 사이 어딘가에 선을 그으면 되지 않을까 싶었습니다.

6-2 다음 실습에서 다룰 것

다음 실습에서는 다음과 같은 것들을 다룹니다.

- **데이터셋 확인** — `hr_sft_all.jsonl`, `hr_sft_train.jsonl`, `hr_sft_validation.jsonl` 을 열어 구성을 먼저 봅니다.
- **LoRA SFT 학습** — `SFT_MAX_LENGTH=384`, `SFT_EPOCHS=2`, `SFT_LEARNING_RATE=1e-4` 로 `scripts/train_lora.py` 를 실행해 `models/hr-qwen-lora/` 에 Adapter 를 만듭니다.
- **성능 평가** — `scripts/evaluate_model.py` 로 Base 모델과 SFT 모델을 Keyword · F1 · Hallucination 세 지표로 비교합니다.
- **재학습** — 다시 학습할 때는 `rm -rf models/hr-qwen-lora` 로 기존 Adapter 를 지우고 시작합니다.
- **데모 사이트 확인** — Base Model 과 SFT Model 을 비교하고, RAG 를 붙였을 때와 아닐 때의 차이도 함께 확인합니다.

아래 의문들은 SFT 모델이 나와야 답할 수 있는 것들이라, 위 과정을 돌리면서 확인해 보려고 합니다.

- 수료율 70% 오답이 어느 단계의 문제인지. 숫자 비교 자체를 못 하는 건지 비교는 되는데 결론으로 점프하는 건지 구분하고 싶은데, Base 와 SFT 를 나란히 놓고 봐야 뚜렷해질 것 같습니다.
- `lora_alpha` 는 학습되는 값이 아닙니다. 코드에서는 `r=8, lora_alpha=16` 으로 그냥 넣고 지나갔는데 찾아보니 실제 반영량은 `alpha / r` 배입니다. 그러면 학습을 마친 뒤에 alpha 만 바꿔서 불러오면 SFT 세기를 조절할 수 있는지 궁금합니다.
- `q,v / r=16` 과 `q,k,v,o / r=8` 은 파라미터 수가 같은데 성능도 같은지. 오늘은 숫자만 확인했고 학습은 안 해 봤습니다.
- 582건 중 거절이 6건, 1% 인데 평가셋은 20건 중 5건, 25% 가 거절 문제입니다. 학습은 1% 만 보고 평가는 25% 로 하는 게 이상하다고 느꼈습니다.
- Keyword Score 와 Token F1 은 문자열이 겹치는지로 채점합니다. SFT 모델은 학습 답변과 형식이 같으니 구조적으로 유리할 텐데, 그러면 점수가 오른 게 지식을 얻어서인지 형식을 흉내 내서인지 어떻게 구분하나 싶습니다.

부록. 검색 점수로 Knowledge Base 밖 질문을 걸러낼 수 있을까

> 2장을 정리하다 걸렸던 질문 하나를 오늘 실습 환경에서 확인해 봤습니다. 가설을 먼저 세우고 방법을 정한 뒤 돌렸는데 가설이 예상과 달랐고, 틀린 부분도 그대로 남겨 뒀습니다.

부록-1 궁금했던 점

2장에서 검색 점수를 정리하다 보니 Knowledge Base 안 질문은 0.69 에서 0.81, Knowledge Base 밖 항공료 질문은 0.56 이었습니다. 그 사이 어딘가에 선을 하나 그으면 되는 것 아닌가 싶었습니다.

지금 코드는 점수가 몇이든 무조건 Top-2 를 넣고, 거절할지 말지는 Qwen 이 System Prompt 를 지켜 주기만 바라는 구조입니다. FAISS 가 이미 점수로 신호를 주고 있으니 검색 단계에서 먼저 잘라 내면 더 안전하지 않을까 싶었습니다.

부록-2 가설과 방법

두 분포 사이에 τ ≈ 0.62 로 선을 그으면 Knowledge Base 밖 질문은 전부 막으면서 Knowledge Base 안 질문은 하나도 막지 않을 것이다.

2장의 숫자는 질문 4개로 얻은 거라 표본이 적다는 게 걸려서 질문 수를 늘려 확인했습니다. Knowledge Base 안 질문 12개와 Knowledge Base 밖 질문 12개를 만들고 각각 Top-1 점수를 기록한 뒤, τ 를 0.45 에서 0.85 까지 0.01 씩 옮기면서 Knowledge Base 밖을 얼마나 막는지와 Knowledge Base 안을 잘못 막는 비율을 계산했습니다.

exp_a1.py · Cell 실험 1 · KB 안 12문항 / KB 밖 12문항의 Top-1 score

In [1]:

```
IN_KB  = [ ... KB 안 질문 12개 ... ]   # 규정 6건으로 답할 수 있는 질문
OUT_KB = [ ... KB 밖 질문 12개 ... ]   # 연차 / 야근수당 / 출산휴가 등

rows = []
for q in IN_KB:  rows.append({"q": q, "in_kb": True,  "top1": search(q, 1)[0]["score"]})
for q in OUT_KB: rows.append({"q": q, "in_kb": False, "top1": search(q, 1)[0]["score"]})
```

※ 질문 리스트 본문은 지면상 생략했습니다.

Out:

```
KB 안  n=12  min=0.5460  max=0.8096  mean=0.7089
KB 밖  n=12  min=0.4037  max=0.5514  mean=0.4705

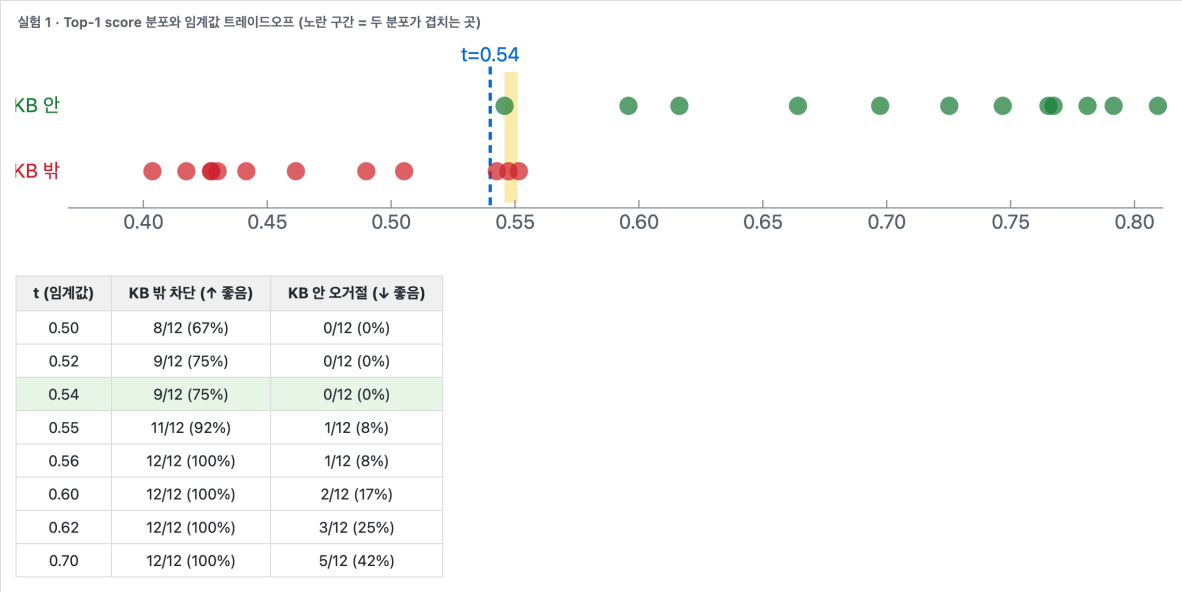
-- KB 안, 점수가 낮은 순 --
  0.5460  집에서 일해도 괜찮은가요?
  0.5959  교육비 지원 제도의 코드명이 뭔가요?
  0.6164  사전 승인 없이 들은 교육도 지원되나요?
  0.6643  재택근무를 하려면 누구의 승인이 필요한가요?

-- KB 밖, 점수가 높은 순 --
  0.5514  연차는 며칠까지 쓸 수 있나요?
  0.5473  출산휴가는 며칠인가요?
  0.5428  해외 AI 컨퍼런스 항공료를 전액 지원하나요?
  0.5055  야근 수당은 시간당 얼마인가요?

두 분포가 겹칩니다.  KB안 최저 0.5460  <  KB밖 최고 0.5514
```

실험 1 코드와 score 분포 출력

부록-3 결과



t (임계값)	KB 밖 차단 (↑ 좋음)	KB 안 오거절 (↓ 좋음)
0.50	8/12 (67%)	0/12 (0%)
0.52	9/12 (75%)	0/12 (0%)
0.54	9/12 (75%)	0/12 (0%)
0.55	11/12 (92%)	1/12 (8%)
0.56	12/12 (100%)	1/12 (8%)
0.60	12/12 (100%)	2/12 (17%)
0.62	12/12 (100%)	3/12 (25%)
0.70	12/12 (100%)	5/12 (42%)

Top-1 score 분포와 임계값 트레이드오프

> 두 분포가 겹쳤습니다. Knowledge Base 안 최저가 0.5460 인데 Knowledge Base 밖 최고가 0.5514 였습니다.
>
> 겹치는 구간이 있으니 Knowledge Base 안과 밖을 완벽히 가르는 τ 는 없습니다. 질문 4개로 봤을 때는 0.13 이나 벌어져 보였는데 12개씩으로 늘리니 간격이 사라졌습니다.

제가 예상했던 τ=0.62 를 실제로 적용하면 어떻게 되는지 확인해 봤습니다.

τ	Knowledge Base 밖 차단	Knowledge Base 안 오거절	해석
0.54	9/12 (75%)	**0/12 (0%)**	답할 수 있는 질문은 하나도 막지 않는 최대 지점
0.56	**12/12 (100%)**	1/12 (8%)	Knowledge Base 밖을 다 막으려면 오거절 1건을 감수해야 함
0.62 (내 예상)	12/12 (100%)	**3/12 (25%)**	답할 수 있는 질문 4개 중 1개를 잘못 막음
0.70	12/12 (100%)	5/12 (42%)	절반 가까이 막힘

부록-4 점수가 애매하게 나온 질문들

어떤 질문이 경계에 걸렸는지 보다가 예상 못 한 걸 발견했습니다.

> Knowledge Base 안에서 점수가 가장 낮았던 질문은 "집에서 일해도 괜찮은가요?" 로 0.5460 이었습니다. 2장에서 키워드가 안 겹쳐도 뜻이 비슷하면 찾아온다며 Semantic Search 의 장점으로 소개했던 바로 그 질문입니다. 검색은 재택근무 규정을 정확히 찾아왔는데 표현이 많이 달라서 점수는 제일 낮게 나왔습니다. 의역한 질문일수록 점수가 낮으니 임계값을 올리면 이런 질문부터 잘려 나갑니다.
>
> Knowledge Base 밖에서 점수가 가장 높았던 건 "연차는 며칠까지 쓸 수 있나요?" 로 0.5514 였습니다. 규정에 연차 조항이 없는데도 점수가 높았는데, 리프레시 휴가 문서와 뜻이 가깝기 때문입니다. 그다음도 "출산휴가는 며칠인가요?" 로 0.5473, 역시 휴가 계열이었습니다.
>
> 그러니까 점수는 뜻이 얼마나 가까운지를 재는 값이지 답이 여기 있는지를 재는 값이 아니었습니다. 인접한 주제일수록 답이 없어도 점수가 높게 나오니, 제가 임계값에 기대했던 역할을 애초에 점수가 할 수 없는 구조였습니다.

부록-5 알게 된 것

임계값은 환각을 없애는 스위치가 아니라 안전하게 갈지 많이 답할지를 정하는 운영 값이었습니다. 틀린 답의 대가가 큰 HR 챗봇이라면 τ 를 0.56 으로 두고 오거절 8% 를 감수하는 편이 낫고, 응답률이 중요하면 0.54 로 두면 됩니다.

실제로 쓴다면 막힌 질문을 그냥 버리지 말고 "관련 규정을 찾지 못했습니다, 혹시 이 중에 있나요" 하면서 Top-3 제목을 같이 보여 주면 1장에서 정리한 재확인 비용을 줄일 수 있겠다고 생각했습니다.